

Billie — Personal AI Agent — Postmortem

Status: STOPPED on 25 September 2026. **Repo folder:** `D:\Workspace\personal-ai-agent-arch` (Windows, VS Code + PowerShell). **Alt code name during early chats:** ARCH. **Duration of active build:** roughly 2 weeks of deep work spread across Chats 03 → 06 (12 Sept → 24 Sept 2026). **Author:** Raghav Vohra. Teaching/pair partner: Claude. Reviewer: Abhishek.

This document exists so that when I revisit this project six months from now, I can reconstruct **exactly what I built, why I built it, what I learned, what I got wrong, and what carries forward**. Nothing is hidden. This is not a "shipping" README — it is a graveyard log written on purpose so that the graveyard becomes a foundation, not a hole.

0. TL;DR

I set out to build **Billie**, a blended life + career coach personal AI agent, from scratch, in Python, atomically, one component at a time. I completed **7 of 8 planned steps** — a full working end-to-end conversational agent with memory, persona, drift-check, mood, multi-layer safety, and an orchestrator wired into a live CLI chat loop. The engineering was solid.

I then **stopped the project** because two structural gaps became impossible to ignore:

1. **No product vision was ever locked.** I built the machine before I understood what the machine was for.
2. **No cost/model discipline.** I picked `gpt-4o-mini`/`gpt-4o` by reflex, never audited alternatives, never calculated per-message cost.

The engineering learnings — memory contradiction resolution, Hinglish safety gaps, multi-label crisis handling, orchestrator design, fail-closed philosophy, eval-first discipline — are **directly transplantable** to the next project (mini-Hermes). The product is dead. The learning is not. That is the frame.

1. What I set out to build

- A standalone personal AI agent named **Billie**, functioning as a blended **life + career coach**.
- Interface: text + voice (cascaded pipeline STT → text agent → TTS, chosen over real-time voice APIs for cost).
- Storage: local-first V1 (SQLite + local Qdrant), hybrid cloud as a potential V2.

- Interaction mode: both **proactive and reactive**, configurable.
- Tone: casual Hinglish, blended-friend feel.
- Named "Billie" with an intentional relational-friend positioning, not a task-bot.
- Not integrated into the QA portfolio — a standalone personal project.
- Threat model in V1: single-user local machine, so no encryption-at-rest.
- Deferred to V2: encryption-at-rest, cloud sync, voice pipeline.

2. Timeline — the four chats that built this

Chat	Date range	What happened
CHAT 03	~12 Sept 2026	Deep-study session. Seven core components researched: Mem0, Letta/MemGPT, Zep/Graphiti, ReAct, Safety (Ben-Zion + Joiner), Cascaded Voice, Dependency-Safe Design. 8-phase atomic build sequence locked.
CHAT 04	~12 Sept 2026	Major research finding: Mem0's new OSS release changed to single-pass ADD-only extraction — no more UPDATE/DELETE/NOOP. This forced the custom Confidence/Decay/Superseded layer from "optional polish" to "sole mechanism for contradiction resolution." Step 1 (storage) fully complete. Step 2 (Mem0 core add/search) verified.
CHAT 05	~20 Sept 2026	Steps 3, 4, 5 built and tested (Persona, Drift-check, Mood). <code>ARCHITECTURE.md</code> produced for sharing with Abhishek.
CHAT 06	20 → 24 Sept 2026	Long, intense session. Step 6 (Safety, five sub-pieces + Gate 1b) and Step 7 (Orchestrator + <code>chat_loop.py</code>) built and live-tested. Abhishek delivered blunt critique (no vision, no cost analysis, no model rigor). Two "Baithak" sessions planned (Vision & Scope; Model & Cost audit). Baithak 1 opened but never closed — vision fractured between "emotional witness" and "learning coach".
CHAT 07 (this one)	25 Sept 2026	Decision to STOP. Postmortem written. Pivot planned to mini-Hermes.

3. Roadmap — what got built, what didn't

Step	Piece	Status
1	Foundation storage (SQLite + Qdrant local + <code>text-embedding-3-small</code>)	✅ Complete, tested in isolation and combined
2	Memory (Mem0 core + custom Confidence/Decay/Superseded layer)	✅ Complete, contradiction resolution proven
3	Persona (Billie, hand-rolled Letta/MemGPT two-block pattern)	✅ Complete
4	Drift-check (persona-consistency judge, one-shot repair, every 8 turns)	✅ Complete, root-cause bug found and fixed
5	Mood layer (Ekman emotion detection, current-vs-past distinction)	✅ Complete
6	Safety layer (5 sub-pieces + later Gate 1b)	✅ Complete, live-tested, regression-locked
7	Orchestrator (<code>generate_reply.py</code> , <code>orchestrator.py</code> , <code>chat_loop.py</code> , debug mode)	✅ Complete, live end-to-end working
8	Voice (STT → agent → TTS cascaded pipeline)	❌ Never started
Deferred	Gate 2b (mood-history context for Gate 2)	⏸ In backlog

4. What I built — step by step, with every decision

Step 1 — Storage foundation

What. Local storage plumbing: SQLite (for structured facts/decisions/state) + Qdrant local mode (for vector search) + OpenAI's `text-embedding-3-small` (for embeddings).

Why.

- Local-first because V1 is a single-user machine, no cloud dependency required.
- Qdrant in **embedded/local mode**, not Docker — because Docker was not available on the Windows dev machine. Trade-off: **only one process can access the storage path at a time**.
- `text-embedding-3-small` because it's cheap, fast, and 1536-dim is more than enough for personal-scale memory.

Key decisions.

- Used Qdrant's current `query_points()` API, not deprecated `search()`.
- Central `config.py` at project root holds all paths, model names, and the Mem0 config dict.
- `.env` for the OpenAI key (never committed).
- Four test scripts run in isolation before combining: `test_sqlite.py`, `test_qdrant.py`, `test_embedder.py`, `test_plumbing_combined.py`. **All passed before Step 2 began.**

Learnings.

- Python 3.13.12 broke `mem0ai[nlp]` — spaCy is not yet compatible. **Fallback:** retrieval runs semantic-only with graceful degradation, no crash. This foreshadows the entire "dependency-safe design" principle we later formalised.

Step 2 — Memory layer (Mem0 + custom Confidence/Decay/Superseded)

What. Mem0 handles extraction of atomic facts from raw messages and stores them with embeddings. On top of Mem0 sits a **custom lifecycle layer** that assigns each fact a confidence state and resolves contradictions.

Why the custom layer exists. Mem0's **current OSS release changed to a single-pass ADD-only extraction pipeline**. The older UPDATE/DELETE/NOOP behaviour was removed. This was verified empirically: told Mem0 "QA engineer," then "switched to SDET" — both facts sat side-by-side in memory with no resolution. So Mem0 alone will never contradict-resolve; the custom layer is now the **sole** mechanism for that.

How the lifecycle works (Golden Dataset + Confidence + Decay).

Every extracted fact (not raw message) is classified by a Memory Gate into one of three buckets:

- **Durable** — store it, run it through the confidence lifecycle.
- **Ephemeral** — session-only, discarded after conversation.
- **Never store** — blocked entirely (e.g. sensitive protected attributes).

Confidence lifecycle for Durable facts:

- **Stated** — first mention, tentative confidence.
- **Confirmed** — promoted after a second corroborating mention in a *separate* session.
- **Decayed** — Confirmed → Tentative after 60 days of silence.
- **Superseded** — direct contradiction supersedes immediately, not decayed.
- **Facts and explicit decisions are exempt from decay.**

Gate implementation: rules-first hybrid, LLM fallback only for ambiguous cases (cost-efficient — most facts are unambiguous).

Key decisions.

- Custom Superseded state is **owned by the agent layer**, not delegated to Mem0.
- Golden dataset eval structure: 4 fields per case — `message`, `bucket`, `lifecycle_action`, `reasoning`. Built **before any code**, so the code is scored against a known truth.

Step 3 — Persona (Billie, hand-rolled Letta/MemGPT pattern)

What. Two always-in-context blocks: a static **Persona block** (Billie's identity, tone, boundaries) and a periodically-refreshed **Human block** fed only from Confirmed memories.

Why this pattern.

- The full Letta server was overkill and added a heavy runtime dependency.
- Hand-rolled two-block pattern gives 90% of the benefit with 10% of the surface area.
- Human block is fed **only from Confirmed** (not Stated) memories — this is deliberate: it means Billie's mental model of me updates slowly, avoiding whiplash on one-off mentions.

Key decisions.

- Character-cap on both blocks to prevent context bloat.
- Persona is edited manually in a file (`persona.md` or similar), not learned/mutated by the agent. Explicit and human-controlled.

Step 4 — Drift-check

What. Every 8 turns, an independent LLM judge checks: is Billie still speaking consistently with the Persona block? If drift is detected, a one-shot repair prompt is issued.

Why 8 turns. Research-backed cadence — frequent enough to catch drift early, sparse enough not to hammer costs.

Key bug found and fixed. `drift_checker.py` was **found missing from disk mid-project** — an earlier fixed version had been lost. Rebuilt from prior verified logic. **Learning: no file lives in isolation; every file must be reproducible from its `story-comments` plus its eval cases.**

Step 5 — Mood layer

What. Detects current message mood using Ekman's basic emotions (anger, disgust, fear, happiness, sadness, surprise). Distinguishes **current-message mood** from **past-mood context** — the two are stored separately so a bad-day flare doesn't poison the long-term model.

Why decoupled from Step 2's decay system. Mood is intentionally short-lived and volatile. Decaying it like a preference-fact would either be too slow (mood lasts hours, not months) or would interfere with the confidence lifecycle for genuine preferences.

Step 6 — Safety layer (the biggest, hardest step)

What. Five sub-pieces, plus a later-added Gate 1b:

1. **Disclosure message** — Billie tells the user, on first contact, what she is and is not (an AI, not a therapist; will contact helplines in emergencies; etc.).
2. **Gate 1 — OpenAI Moderation API** — first-pass fail-closed filter for explicit self-harm signals.
3. **Gate 1b — `check_other_harm`** — added after live testing (see bug 5 below).

4. **Gate 2 — custom LLM distress-tiering judge** — 4-tier classification: NONE / MILD / MODERATE / ACUTE.
5. `safety_manager.py` — combines both gates with **OR-escalation** logic. If either gate escalates, the higher tier wins.
6. `safety_responses.py` — **fixed (non-LLM-generated) crisis content** for the ACUTE tier, using verified India national helplines: **Tele-MANAS (14416)** and **KIRAN (1800-599-0019)**. No dynamic generation for crisis text — too risky.

Why fail-closed everywhere. If any safety component errors, the system defaults to the *most conservative* response, not the most permissive. Safety is not a "best effort" concern.

Five bugs found via testing, each with its own root-cause fix:

- **Bug A — Hinglish moderation gap.** Empirically discovered via `debug_moderation_scores.py`. Same-meaning sentence:
 - Hinglish: `Mujhe lagta hai jeena chhod dena chahiye, sab bekar lagta hai ab.` → `self_harm: 0.0802`, `self_harm_intent: 0.0227`.
 - English translation: `I feel like I should stop living, everything feels pointless now.` → `self_harm: 0.7753`, `self_harm_intent: 0.7358`.
 - **10× gap.** Root-cause: OpenAI Moderation API's training data is English-heavy. No single threshold could cover both.
 - **Fix (root-cause, not patch): Translate-then-moderate** — small LLM call (`gpt-4o-mini`) translates message to English first, then moderation runs on the English text. API works on its actual strength, not its blind spot.
- **Bug B — Passive suicidal ideation missed.** Perceived burdensomeness ("everyone would be better off without me") is a clinical marker per Joiner's Interpersonal Theory of Suicide but Gate 2 wasn't catching it. **Fix:** updated Gate 2 tier definitions to explicitly include burdensomeness signals as MODERATE or ACUTE.
- **Bug C — Retraction after ACUTE.** User sends an ACUTE message, then next turn says "I was kidding." Single-message-scoped Gate 2 read the retraction as NEUTRAL and Billie switched to casual mode — catastrophic. **Fix:** orchestrator now tracks `previous_tier`. If previous was ACUTE and current is neutral, return a fixed *gentle check-in* response instead of resuming normal chat.
- **Bug D — Combined self-harm + harm-to-others.** A message containing both signals got only a self-harm response. **Fix:** added Gate 1b (`check_other_harm`), treating self-harm and harm-to-others as **independent dimensions**. Both signals can fire; both get separate response components (rather than resolving which is "true").

- **Bug E — Noisy warnings.** mem0/spaCy/fastembed emitted noise; Qdrant threw a shutdown exception on process exit.
Fix: logging level control + `atexit` cleanup for Qdrant.

Deferred: Gate 2b (mood-history context to further sharpen Gate 2). In backlog, not built.

Step 7 — Orchestrator

What. Three files wire everything into a live chat:

- `generate_reply.py` — the core reply-generation function. Combines persona + retrieved memory + mood into a single LLM call.
- `orchestrator.py` — the full pipeline: `safety-check → mood → memory-retrieve → generate → drift-check/repair → memory-add → disclosure`. Sequential, deterministic order.
- `chat_loop.py` — interactive CLI. This is what makes Billie feel like a real agent.
- **Debug mode** — inspect internals (which memories retrieved, mood tier, safety tier, drift verdict) live.

Why sequential, not parallel. Simplicity for V1; each stage's output feeds the next. Parallelism deferred to V2 if a latency budget forces it.

Live testing surfaced Bugs C and D above (the retraction bug and the combined-harm bug). Both are now regression-locked in the eval suite.

Step 8 — Voice pipeline

Never started. Cascaded design was researched (STT → text agent → TTS chosen over real-time voice for cost) but no code written.

5. Research I did (with actual sources, honestly)

Topic	What I studied	Where it landed in the build
Mem0 (memory framework)	ADD-only pipeline, hybrid retrieval, current OSS behaviour	Step 2 core; forced the custom lifecycle layer
Letta / MemGPT (persona pattern)	Two-block always-in-context design	Step 3, hand-rolled version
Zep / Graphiti (temporal knowledge graph)	Studied for completeness	Rejected — overkill for personal scale
ReAct (orchestration pattern)	Reasoning-Action loop	Informed Step 7 orchestrator design
Ben-Zion et al. — four safeguards for LLM safety	Disclosure, refusal, redirection, escalation	Step 6 architecture
Joiner — Interpersonal Theory of Suicide	Perceived burdensomeness as a clinical marker	Fix for Bug B in Step 6
Ekman — basic emotions taxonomy	6-category emotion set	Step 5 mood layer
Cascaded Voice Pipeline (STT → agent → TTS)	Cost/latency trade-offs vs real-time voice	Step 8 (never built) design
Dependency-Safe Design	Circuit breaker, graceful degradation, fail-open vs fail-closed	Applied throughout, especially spaCy fallback in Step 1 and fail-closed in Step 6
Verified India crisis helplines	Tele-MANAS (14416), KIRAN (1800-599-0019) — validated against government sources	Step 6, <code>safety_responses.py</code> fixed content

Research I did NOT do (this is the honest gap):

- Model comparison across providers (OpenAI vs Anthropic vs Google vs open-weights).
- Reasoning-model evaluation (o1, o3, DeepSeek-R1) — could any of the "judge" calls benefit?
- Per-call cost calculation for any pipeline stage.
- Realistic monthly cost projection at expected usage.
- Alternative embedding models (comparison against `text-embedding-3-small`).

This gap is **exactly what Abhishek called out** and it is the second biggest reason this project stopped. Detailed in Section 7.

6. What I did WELL — engineering practices worth keeping

- **Fail-closed design.** Every safety component defaults to the most conservative response on error.
- **Atomic isolated builds.** Every component tested in isolation *before* integration. No exceptions. Storage plumbing had 4 isolated tests + 1 combined test before Step 2 could begin.
- **Root-cause before patch.** The Hinglish moderation gap is the clearest example — I could have thresholded and moved on, but instead ran `debug_moderation_scores.py`, proved the 10× gap, and translate-then-moderate emerged as the root-cause fix.
- **Eval suites built alongside each component.** Not "test after" — test *with*. Step 2 had a golden dataset before the code was written.
- **Multi-label safety architecture.** Bug D taught me that safety dimensions are independent; treating them as mutually exclusive is unsafe.
- **Verified crisis content, no LLM generation for crisis.** Helplines were validated against real Indian government sources, not model output.
- **Central `config.py`.** Every model name, path, and threshold in one file. Ready for a cost audit that never happened.
- **`DECISIONS.md`** maintained throughout — every non-obvious choice recorded with rationale.
- **Story-comments in code.** Every file readable as a narrative six months later — the reason this postmortem is even possible.

- **Honest teacher-student collaboration.** Pushback and disagreement were welcomed on both sides. Claude questioned me when I drifted; I questioned Claude when explanations felt abstract or when advice was cargo-culted.
-

7. What I LACKED — the honest gaps that killed the project

Two categories. Both explicitly named by Abhishek. Both correct.

7A. No product vision was ever locked

I built the machine before I understood what the machine was *for*. Symptoms:

- **No single-verb identity.** Great products have one verb: Hermes GROWS, Strava TRACKS, Notion CAPTURES. Billie's positioning was "a place where we can be ourselves, be seen, be heard" — **three verbs**, three products, unresolved tension.
- **No scope boundary.** Never wrote down what Billie explicitly does NOT do.
- **No user definition.** "Personal life/career coach" — is the user just me, or a broader audience? Different answers → totally different products.
- **No success criteria.** No answer to "how do I know V1 is done?"
- **No moat.** "What does Billie give me that ChatGPT/Claude/journaling/a therapist doesn't?" — never resolved.
- **Fractured mid-flight into two candidate products** and I couldn't reconcile them:
 - *Emotional witness* — Billie sees me, tracks my small details over time, holds unperformed real space.
 - *Learning coach* — Billie gives research-backed guidance on schedules, revision, deep work, with sources for every claim.
- **No daily-return mechanism.** Discussed Strava-style streaks vs relational pull (Hermes-style async messaging) but never chose. Without a daily-return mechanism, even a beautiful product dies from disuse.

The Baithak 1 session opened this. It never closed. VISION.md draft exists but is not authoritative.

7B. No cost or model discipline

- Model choices were **reflex, not rigor**. `gpt-4o-mini` for everything cheap, `gpt-4o` for anything harder. I never compared providers, never evaluated reasoning-specialist models for the judge/classifier calls, never priced alternatives.

- **Zero cost math.** Never once did I sit down and calculate: 1 user message → N LLM calls → M tokens → ₹X. So I have no idea what Billie actually costs to run, and no idea whether it would be viable at scale.
- **Baithak 2 (Model & Cost audit) was planned. It was never held.**

7C. Learning-goal misalignment (the third gap I only saw at the end)

Despite building 4 agent projects in the QA portfolio (Requirements-to-Test, Self-Healing Locator, UI Testing ReAct, Data-Analysis Verifier) and then 7 steps of Billie, I realised at the end that I still don't feel I understand **how an agent works at the mechanics level** — because I have always used OpenAI's tool-calling wrapper as a black box. I want to build the loop from scratch. Billie was not teaching me that; it was teaching me framework integration (Mem0, safety patterns, orchestration composition) which is *also* valuable but different.

8. What I LEARNED — the transferable takeaways

Engineering learnings (carry forward to every future agent)

1. **Frameworks change; don't build on assumed behaviour.** Mem0 changed its pipeline mid-project and forced a re-architecture. Always verify current behaviour empirically before designing on top.
2. **Contradiction resolution is agent-owned, not framework-owned.** Never delegate this to the memory framework.
3. **Language matters for safety APIs.** English-heavy training data means non-English text under-scores dangerous signals. Translate-then-moderate is a general pattern for any non-English agent.
4. **Safety dimensions are independent, not exclusive.** Multi-label handling from day one.
5. **Retractions and context shifts break single-message classifiers.** Track `previous_tier` and apply time-window rules.
6. **Fixed content beats LLM-generated content for high-stakes responses** (crisis, financial, medical).
7. **Fail-closed everywhere in safety-critical paths.** Errors default to conservative.
8. **Eval suites built alongside code, not after.**
9. **Root-cause > patch.** Always debug first (a 20-line debug script beats 500 lines of workaround).
10. **Atomic builds save your sanity when something breaks 5 steps later.**

Product learnings (carry forward before writing any code for anything)

1. **Vision before code, always.** A product without a single verb is a technology looking for a use case.

2. **Anti-scope is as important as scope.** Naming what you WON'T do prevents feature-creep.
3. **Daily-return mechanism is not optional.** A product with no mechanism to bring users back dies regardless of quality.
4. **Cost math before the first LLM call.** Even a rough per-message estimate would have surfaced problems Abhishek later flagged.
5. **Model selection is a research task, not a default.** Every pipeline stage deserves its own "which model is best for this specific job at this specific cost" answer.
6. **A friend who gives blunt feedback is worth ten friends who nod.** Abhishek's critique was uncomfortable and correct.

Meta-learnings about how I work

1. **I chase engineering elegance before I've earned it with product clarity.** This is a pattern to watch.
2. **I resist "boring" upstream work** (vision doc, cost sheet) in favour of "exciting" downstream work (code). This trade-off has costs I now understand.
3. **I need someone to say "stop" when I'm building on unclear foundations.** Abhishek did that here. Claude tried to do it earlier in Chat 06 (flagged founder-vision-drift) but I didn't hear it until the second time.

9. Salvageable — what carries forward to mini-Hermes and beyond

Concrete code, patterns, and files worth reusing:

Reusable code

- **Storage foundation** (`test_sqlite.py`, `test_qdrant.py`, `test_embedder.py`, `test_plumbing_combined.py`) — plug-and-play plumbing for any local-first agent.
- `config.py` **convention** — central config pattern.
- **Custom Confidence/Decay/Superseded layer** — applies to any memory system that lacks native contradiction resolution.
- **Translate-then-moderate pattern** — any non-English agent that needs moderation.
- `safety_responses.py` **fixed-content pattern** with verified helplines — reusable for crisis flows in any user-facing agent.
- `safety_manager.py` **OR-escalation wiring** — general multi-gate escalation pattern.

- **previous_tier tracking pattern** — any multi-turn classifier that must resist retractions.
- **Multi-label safety architecture** — Gate 1 + Gate 1b independent-dimension pattern.
- **Debug-mode orchestrator** — inspect internals live, invaluable for development.
- **Mood detection code** (Ekman-based) — reusable emotion-tagging.
- **Persona/Letta two-block pattern** — hand-rolled implementation reusable for any agent needing consistent identity.
- **Drift-check judge** — persona-consistency checker with one-shot repair.

Reusable disciplines

- Atomic isolated builds with per-component eval suites.
- Root-cause debugging before patching.
- Story-comments as living documentation.
- **DECISIONS.md** alongside code.
- Fail-closed default in safety-critical paths.

Reusable research

- Ben-Zion safeguards, Joiner burdensomeness, Ekman emotions, ReAct pattern, Letta pattern, dependency-safe design principles.

NOT reusable (mistakes to avoid)

- Model-selection-by-default.
- Skipping cost math.
- Building without a locked vision.
- Assuming "personal project" means "no product discipline required."

10. Why the project stopped — a clean decision log

Date: 25 September 2026. **Decision-maker:** Raghav. **Trigger events:**

1. Abhishek's blunt feedback (24 Sept) — no vision, no cost analysis, no model rigor.

2. Baithak 1 vision session (24 Sept) — could not converge on a single-verb vision. Fractured between "emotional witness" and "learning coach."
3. Deeper realisation (25 Sept) — I still don't understand how an agent works at the mechanics level, despite building 4 QA-portfolio agents + 7 steps of Billie. I want to build the loop from scratch, learn foundations.

Decision: Stop Billie. Pivot to **mini-Hermes** — a self-improving skills-based agent, "GROW" verb, built from more foundational primitives, no timeline pressure, learning-first goal.

What happens to Billie:

- Repo remains public at `personal-ai-agent-arch` with this POSTMORTEM.md prominent.
- Code archived as-is, no further development.
- Learnings, code patterns, and eval suites transplant to mini-Hermes wherever they fit.

What this is NOT:

- Not a failure of the engineering work. The engineering was solid.
- Not a failure of the research. The research was substantial and correct.
- **A failure of upstream discipline** — vision and cost were not treated as first-class engineering concerns.

What this IS:

- A completed learning cycle.
- A functional, working V1 agent that lives in the repo as proof of the engineering.
- A clear-eyed handoff document (this file) so the learnings compound instead of evaporating.

11. Appendix — files, keys, dependencies

Project root: `D:\Workspace\personal-ai-agent-arch`

Key files:

- `config.py` — central config

- `.env` — OpenAI API key (never committed)
- `ARCHITECTURE.md` — architecture doc (built for Abhishek in CHAT 05)
- `DECISIONS.md` — running decision log
- `VISION.md` — draft, unfinished (see Section 7A)
- Step-wise folders and test scripts

Storage:

- SQLite database (local)
- Qdrant local mode (single-process access)

External dependencies:

- `mem0ai==2.0.20`
- `qdrant-client>=1.12.0`
- `openai`
- `python-dotenv`
- `tenacity`
- Python 3.13.12 (blocks spaCy — graceful semantic-only fallback in place)

Models used (with the cost-discipline caveat):

- Generation: `gpt-4o` and `gpt-4o-mini` (reflex choice, never audited)
- Embeddings: `text-embedding-3-small`
- Safety judge: `gpt-4o-mini`
- Translation (for moderation): `gpt-4o-mini`

Verified helplines used in crisis responses:

- Tele-MANAS: 14416
 - KIRAN: 1800-599-0019
-

This postmortem is deliberately long. When I return to this repo months from now, I want the full story — decisions, mistakes, and learnings — reconstructable from this one file. If you are reading this and something is unclear or missing, add it. The graveyard should keep growing until it becomes a foundation.

— Raghav Vohra 25 September 2026